

VibeMeGood — Replit Agent Instructions

The single document to read before writing a single line of code.

Attach alongside this document:

- `VibeMeGood-ReplitSpec.md` — full implementation reference with schemas, route code, projection models, weather, SSE, and optimizer logic
- `VibeMeGood-main.zip` — existing codebase

Read this document first. Then read the spec. Then build.

WHAT THIS IS

A decision intelligence terminal disguised as a PrizePicks analytics tool. The user's edge comes from being systematic where their competition is emotional. Every feature exists to either find an edge, quantify it precisely, or protect the user from destroying it through bad behavior. If a feature does not do one of those three things, it should not be in this app.

The primary long-term proof metric is **Closing Line Value (CLV) trending positive over 90+ days**, supported by disciplined staking, source transparency, probability calibration, and behavioral guardrails.

STACK — DO NOT CHANGE

- **Frontend:** React, Wouter, Tailwind, shadcn/ui, TanStack Query
- **Backend:** Express.js, TypeScript
- **Database:** PostgreSQL, Drizzle ORM
- **Auth:** Clerk
- **AI:** Anthropic Claude — server-side only, never expose key to frontend
- **Real-time:** Server-Sent Events (SSE)
- **Scheduler:** node-cron

- **Hosting:** Replit

REQUIRED SECRETS

Add all of these to Replit Secrets before running anything. The app must check for missing keys at startup and log a plain-text warning — do not use emoji in startup logs, they can mangle in deployment environments:

DATABASE_URL	PostgreSQL connection string
CLERK_SECRET_KEY	Clerk backend key
CLERK_PUBLISHABLE_KEY	Clerk frontend key
ANTHROPIC_API_KEY	Claude API key - server-side only
ODDS_API_KEY	the-odds-api.com - free tier, 500 req/month
OPENWEATHER_API_KEY	openweathermap.org - free tier, 1000 req/day

Startup key check (plain text only):

```
const REQUIRED_KEYS = ["DATABASE_URL", "ANTHROPIC_API_KEY", "ODDS_API_KEY", "OPEN
for (const key of REQUIRED_KEYS) {
  if (!process.env[key]) {
    console.warn(`Missing env var: ${key} - related features will be disabled`);
  }
}
```

Missing `ODDS_API_KEY` disables external odds sync but must not crash the server. Missing `OPENWEATHER_API_KEY` disables weather but must not crash the server.

DATABASE — NEW TABLES REQUIRED

Add and migrate all of these without removing any existing working table. Every table must have appropriate unique constraints declared in the schema definition — not as comments, as actual `uniqueIndex` declarations — because `onConflictDoUpdate` targets must reference real column objects.

player_id_mapping	- Links PP player names to external API numeric IDs
our_projections	- Output of our weighted projection model
probability_calibration	- Self-calibrating hit rates per sport/stat/lineType/e
clv_records	- Closing line value per resolved pick
player_streaks	- Consecutive over/under run per player+stat
game_environment	- Weather, pace, and environment score per game

<code>matchup_history</code>	- Head-to-head history per player+opponent+stat
<code>user_settings</code>	- Persisted per Clerk user ID
<code>session_logs</code>	- Daily session tracking for loss limit enforcement
<code>behavioral_logs</code>	- Entry-level behavior tracking for tilt detection
<code>line_move_events</code>	- Book-level line movement history
<code>sync_runs</code>	- Job name, status, records processed, started/finished

The `sync_runs` table is required for source health tracking. `marketDataStatus` on the market intel route depends on knowing whether odds sync has ever run and when. Do not use `dataPullLogsTable` alone — the new `sync_runs` table must be queryable by job name with clear status and timestamps.

Full schema details and unique index declarations are in `VibeMeGood-ReplitSpec.md`.

CONFIRMED BUGS — FIX BEFORE ANYTHING ELSE

Every item in this section will cause a runtime crash, silent data corruption, or features that appear to work but do not.

Bug 1: Internal `fetch()` calls inside the backend

The results route batch handler calls its own API via relative HTTP:

```
// BROKEN — Express cannot resolve relative URLs from within itself
await fetch(`/api/results/pick/${p.pickId}`, { method: "POST", ... });
```

Extract all shared logic into service functions and call them directly:

```
// artifacts/api-server/src/lib/services/result-service.ts
export async function resolvePickResult(
  pickId: number,
  result: "hit" | "miss" | "dnp",
  closingLine?: number
): Promise<void> {
  // All pick resolution logic here — update pick, record CLV,
  // recalibrate probability, update streak, update entry status
}

// In the batch route, call the service directly:
for (const p of picks) {
  await resolvePickResult(p.pickId, p.result, p.closingLine);
}
```

```
}
```

Audit the entire codebase. Every `fetch('/api/...')` call inside Express routes must be replaced with a direct service function call.

Bug 2: resolvePickResult must use DB transactions

Pick resolution touches multiple tables: `entry_picks`, `clv_records`, `probability_calibration`, `player_streaks`, `entries`. If any step fails mid-way, history is corrupted — a pick marked HIT with no CLV record, or calibration updated without the pick result saved.

Wrap in a Drizzle transaction:

```
export async function resolvePickResult(pickId, result, closingLine) {
  await db.transaction(async (tx) => {
    // 1. Update pick result
    await tx.update(entryPicksTable).set({ result }).where(eq(entryPicksTable.id,

    // 2. Record CLV if closing line provided
    if (closingLine) {
      await tx.insert(clvRecordsTable).values({ ... });
    }

    // 3. Recalibrate probability
    await recalibrateProbability(tx, pickId, result);

    // 4. Update player streak
    await updatePlayerStreak(tx, pickId, result);

    // 5. Check if all picks for this entry are resolved; update entry result
    await maybeResolveEntry(tx, entryId);
  });
}
```

If the transaction fails, all changes roll back. No partial state.

Bug 3: All result, CLV, settings, behavioral, and entry routes must scope by Clerk user ID

Every route that reads or writes user data must filter by the authenticated user. No user should be able to read or modify another user's picks, entries, settings, or results.

```
// Pattern for every protected route:
router.post("/results/pick/:pickId", async (req, res) => {
  const userId = req.auth?.userId;
  if (!userId) return res.status(401).json({ error: "Unauthorized" });

  // Verify this pick belongs to this user before allowing modification
  const [pick] = await db
    .select()
    .from(entryPicksTable)
    .innerJoin(entriesTable, eq(entryPicksTable.entryId, entriesTable.id))
    .where(
      and(
        eq(entryPicksTable.id, Number(req.params.pickId)),
        eq(entriesTable.userId, userId) // must match Clerk user
      )
    );

  if (!pick) return res.status(403).json({ error: "Forbidden" });
  // ... rest of handler
});
```

Apply this pattern to: all `/results/*`, all `/entries/*`, `/user-settings`, `/behavioral-logs`, `/session-logs`, `/clv-records`.

Bug 4: Placeholder conflict targets in `onConflictDoUpdate`

Every `onConflictDoUpdate` must reference real column objects, not comments:

```
// BROKEN — Drizzle throws at runtime
.onConflictDoUpdate({ target: [/ * composite unique key */], set: values });

// CORRECT — real column references
.onConflictDoUpdate({
  target: [
    probabilityCalibrationTable.sport,
    probabilityCalibrationTable.statType,
    probabilityCalibrationTable.lineType,
    probabilityCalibrationTable.edgeBucket,
    probabilityCalibrationTable.direction,
  ],
  set: { hitCount: newHits, sampleSize: newSample, hitRate: newHitRate.toString()
});
```

Required conflict targets for every upsert:

- `probability_calibration` → `(sport, statType, lineType, edgeBucket, direction)`
- `our_projections` → `(playerId, statType)`
- `player_id_mapping` → `(playerId, source)` — use `onConflictDoNothing()`
- `external_lines` → `(ppLineId, bookName)`
- `game_environment` → `(gameId)`
- `matchup_history` → `(playerId, opponentTeamId, statType)`
- `player_streaks` → `(playerId, statType)`
- `user_settings` → `(userId)`
- `session_logs` → `(userId, sessionDate)`

Every unique constraint used as a conflict target must be declared as a `uniqueIndex` in the table definition.

Bug 5: Validate all request payloads with Zod

Bad input must return 400, not write garbage to the database. Add Zod validation to every route that accepts a request body:

```
import { z } from "zod";

const PickResultSchema = z.object({
  result: z.enum(["hit", "miss", "dnp"]),
  closingLine: z.number().positive().optional(),
});

router.post("/results/pick/:pickId", async (req, res) => {
  const parsed = PickResultSchema.safeParse(req.body);
  if (!parsed.success) return res.status(400).json({ error: "Invalid input", issu
  // proceed with parsed.data
});
```

Apply Zod validation to:

- `POST /results/pick/:pickId` — result + closingLine
- `POST /results/entry/:entryId` — array of picks with results

- `PUT /user-settings` — full settings object
 - `POST /entries` — entry creation
 - `POST /entry-picks` — pick creation
 - `POST /optimizer/run` — strategy, pickCount, entryType, stake, settings
 - `POST /sync/*` — no body required, but validate query params if used
-

Bug 6: PP sync never deactivates stale lines

After upserting all lines from the latest PP response, lines no longer in that response remain `isActive: true` indefinitely. Old props appear live forever.

Add deactivation after the sync loop:

```
// Track every line ID seen in this sync
const seenLineIds = new Set<number>();

// ... sync loop that populates seenLineIds ...

// After loop: deactivate everything not seen
const allActive = await db
  .select({ id: ppLinesTable.id })
  .from(ppLinesTable)
  .where(eq(ppLinesTable.isActive, true));

const staleIds = allActive.map(l => l.id).filter(id => !seenLineIds.has(id));

if (staleIds.length > 0) {
  await db.update(ppLinesTable)
    .set({ isActive: false, updatedAt: new Date() })
    .where(inArray(ppLinesTable.id, staleIds));
}
```

Bug 7: Player ID mapping never runs on first deploy

The cron triggers it on Sunday at 5 AM. On first deploy, zero mappings exist, so zero projections ever generate. Add a startup check:

```
// In index.ts, before startCronJobs():
async function onStartup() {
```

```
const existing = await db.select().from(playerIdMappingTable).limit(1);
if (!existing.length) {
  console.log("First run: building player ID mappings for all sports...");
  await buildAllPlayerIdMappings();
  console.log("Player ID mapping complete.");
}
}
onStartup().catch(console.error);
startCronJobs();
```

Bug 8: NBA Stats API requires specific browser headers

Without these exact headers, `stats.nba.com` returns a 403 or an empty `resultSets` array with no error thrown. Check both the HTTP status and the response shape:

```
const NBA_HEADERS = {
  "User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.",
  "Referer": "https://www.nba.com/",
  "Origin": "https://www.nba.com",
  "Accept": "application/json, text/plain, */*",
  "Accept-Language": "en-US,en;q=0.9",
  "x-nba-stats-origin": "stats",
  "x-nba-stats-token": "true",
};

const data = await res.json();
if (!data?.resultSets?.length) {
  throw new Error(`NBA Stats API returned empty resultSets for player ${nbaPlayer}`);
}
```

IMPLEMENTATION GAPS — FEATURES THAT MUST NOT BE SKIPPED

Gap 1: Market data must show explicit source health states — never silent null

When no external odds exist, the UI must distinguish between states. Never show a calculated edge when source health is unknown:


```
// On every prop in /api/market-intel response:
const lastOddsRun = await db
  .select()
  .from(syncRunsTable)
  .where(eq(syncRunsTable.jobName, "external-odds"))
  .orderBy(desc(syncRunsTable.finishedAt))
  .limit(1);

marketDataStatus:
  extLines.length >= 2 ? "available" :
  extLines.length === 1 ? "partial" :
  lastOddsRun.length ? "unavailable" :
  "not_synced"
```

In the UI:

- Green dot: available, under 30 minutes old
- Yellow dot: partial or 30–60 minutes old
- Red dot: unavailable or over 60 minutes old
- Grey dot: never synced

When `marketDataStatus` is "unavailable" or "not_synced", do not display a combined edge score. Show: "Line type signal only — no market data." Edge scores derived purely from guesswork are worse than no edge score.

Gap 2: Correlation must be first-class — not a 0.02 scalar

Replace:

```
// INADEQUATE
const stackPenalty = Object.values(teamCounts).filter(c => c >= 2).length * 0.02;
```

With pair-level correlation evaluation:

```
function getCorrelation(p1: any, p2: any): { type: string; severity: "block" | "w
  // NBA/WNBA: same team = negative strong (share possessions)
  if (["NBA", "WNBA"].includes(p1.sport) && p1.team === p2.team) {
    return { type: "negative", severity: "warn", reason: `${p1.team} teammates sh
  }
  // NFL: QB + same-team WR = positive (production linked)
  if (p1.sport === "NFL" && p1.team === p2.team) {
    const isQB = (p: any) => p.statType?.toLowerCase().includes("passing");
```

```

const isWR = (p: any) => p.statType?.toLowerCase().includes("receiv");
if ((isQB(p1) && isWR(p2)) || (isWR(p1) && isQB(p2))) {
  return { type: "positive", severity: "boost", reason: "QB + receiver – prod
}
// Two WRs same team = negative (target share)
if (isWR(p1) && isWR(p2)) {
  return { type: "negative", severity: "warn", reason: "Same-team receivers c
}
}
// MLB: pitcher K prop + same-team batter hits = conflict
if (p1.sport === "MLB" && p1.team === p2.team) {
  const isPitcherK = (p: any) => p.statType?.toLowerCase().includes("strikeout"
  const isBatterHit = (p: any) => ["hits", "total bases"].includes(p.statType?.t
  if ((isPitcherK(p1) && isBatterHit(p2)) || (isBatterHit(p1) && isPitcherK(p2)
    return { type: "conflict", severity: "block", reason: "Pitcher strikeouts c
  }
}
return { type: "neutral", severity: "none", reason: "" };
}

```

- "block" severity: prevent submission of this combination in Power plays. Show an error.
- "warn" severity: show a warning, adjust EV downward by 10–15%, allow submission.
- "boost" severity: adjust EV upward slightly, surface as a positive signal.

Show a correlation matrix in the Entry Builder for every pair of active picks.

Gap 3: Probability source must be labeled on every display

Every probability percentage in the UI must show where it came from. Never show a bare number:

Label	When to Use
Default Estimate	Using the hardcoded bucket from settings
Goblin Line	Derived from line type classification
Market Edge	Derived from True Edge calculation
Your Data (N picks, ±X%)	Calibrated from user's logged results
Combined	Weighted blend of multiple signals

Show sample size and 95% confidence interval as soon as calibration data exists. If sample size is under 30, label as low confidence. If under 10, show an amber warning: "Too few picks to trust — using default estimate."

Gap 4: Complete AI Analyst — all four modes

The mandate document was truncated before specifying these. They are required.

Mode: `analyst` (default) Inject live slate from DB including our projections, market edges, action tags, injury status. Include user's current entry with EV and probability chain. Instruct: lead with the most actionable finding, never recommend Demon OVER, always flag GTD players in Power plays, cite EV in every lineup evaluation, label what is known vs. estimated.

Mode: `anomaly` Prompt: Scan today's full slate. Identify props where True Edge exceeds 10% and explain the probable cause. Flag props that look attractive on paper but carry hidden risk. Note significant line movements and their probable source. Surface hidden correlations among the highest-edge plays. Summarize today's slate quality in three sentences. Name players and numbers — no generic advice.

Mode: `contradiction` Prompt: Check the current plan for contradictions and hidden dangers. Scan for: PLAY-tagged props with GTD or OUT players, teammate stacks in top plays with correlation risk, picks where line type signal contradicts the direction, picks with multiple opposing signals. Format each finding: [RISK TYPE] — Player/Pick — Explanation — Recommendation.

Mode: `stresstest` Prompt: Stress-test this entry before the user locks it in. Find: the single most likely pick to fail and exactly why, any hidden correlation between picks, the true combined win probability accounting for correlation. Recommend: submit as-is, swap one pick, or rebuild. If rebuild, state exactly which pick to remove and what to replace it with. Be direct. Do not soften anything. This person is about to commit money.

All four modes must receive live slate from the database, not hardcoded context. Include current entry picks with EV and probability chain. Include injury states for any player in the entry.

Gap 5: Missing pages — all required

Streak Tracker (`/streaks`) Query `player_streaks` . Show players on a 4+ game consecutive OVER or UNDER run. Columns: Player, Team, Sport, Stat, Streak count and direction, Today's line. Sort by streak length descending. Quick-add to entry.

CLV Tracker (/clv) Query `clv_records` joined with picks and lines. Show: Pick, Locked Line, Closing Line, CLV, Result. Summary: 7-day, 30-day, and 90-day average CLV. CLV trend chart (30 days). Breakdown by sport and stat type.

Include behavioral discipline messages:

- Losing streak + positive CLV: "Your process is sound. This is variance, not a broken system. Do not change strategy."
- Winning streak + negative CLV: "Recent wins may be luck. The market moved against your picks after you locked them. Reassess your pick selection process before increasing stakes."

Matchup Analysis (/matchup) Query `matchup_history` . For each player on today's slate, show career performance vs. tonight's opponent. If no history exists: "Insufficient data — this page populates as you log results over time."

User Guide (/guide) Embed the full VibeMeGood User Guide content as a styled, scrollable page with left sidebar table of contents. Set as default landing page on first visit using `localStorage.getItem('hasSeenGuide')` . After first view, set `localStorage.setItem('hasSeenGuide', 'true')` .

Gap 6: Behavioral logging on every entry submission

```
// Log immediately after entry is saved
await db.insert(behavioralLogsTable).values({
  userId,
  entryId: newEntry.id,
  timeOfDay: getTimeOfDay(), // morning | afternoon | ev
  minutesSinceLastLoss: await getMinutesSinceLastLoss(userId),
  deviatedFromOptimizer: !picksMatchLastOptimizerResult,
  picksChangedFromOptimizer: countPickDifferences(picks, lastOptimizerResult),
  stakeMultipleOfUnit: stake / userSettings.unitSize,
  secondsToDecision: Math.floor((Date.now() - sessionStartTime) / 1000),
});
```

After logging, check for tilt patterns and broadcast via SSE:

```
// Tilt: entry placed within 15 minutes of a loss
if (minutesSinceLastLoss !== null && minutesSinceLastLoss < 15) {
  broadcast("tilt_warning", {
    message: `You placed this entry ${minutesSinceLastLoss} minutes after your la
    severity: "warning"
```

```

    });
}

// Escalation: stake 2x or more above recent average
const recentAvgStake = await getRecentAverageStake(userId, 10);
if (recentAvgStake > 0 && stake >= recentAvgStake * 2) {
  broadcast("stake_escalation", {
    message: `This stake (${stake}) is ${(stake / recentAvgStake).toFixed(1)}x y
    severity: "warning"
  });
}

```

IMPLEMENTATION ORDER — DO NOT SKIP STEPS

1. Run all DB migrations — new tables + unique indexes
 2. Add all secrets to Replit Secrets
 3. Add startup key validation (plain text warnings)
 4. Add startup first-run mapping trigger
 5. Fix all onConflictDoUpdate placeholder targets
 6. Add Zod validation to all input routes
 7. Add Clerk user scoping to all protected routes
 8. Implement syncPpLines() with stale line deactivation
Verify: /api/market-intel returns real active props
 9. Implement syncExternalOdds() with line move event recording
Verify: at least 2 books showing on multiple props
 10. Fix NBA Stats API headers + empty resultSets check
 11. Implement full projection sync (NBA -> NHL -> MLB -> NFL)
Verify: OUR PROJ column shows values with confidence badges
 12. Implement weather enrichment
Verify: outdoor MLB game shows weather badge
 13. Implement real edge scoring with marketDataStatus states
Verify: no edge score shown when market data unavailable
 14. Fix correlation engine — pair-level, not scalar
Verify: NBA teammate stack shows correlation warning
 15. Wrap resolvePickResult in DB transaction
 16. Implement results + CLV + calibration + streaks
Verify: marking a pick updates all 4 tables atomically
 17. Wire all 4 AI Analyst modes with live DB context
 18. Build: Streak Tracker, CLV Tracker, Matchup, User Guide
 19. Implement behavioral logging + tilt/escalation detection
 20. Add calibration debug endpoint (see acceptance test below)
 21. End-to-end test: all 14 acceptance criteria
-

ACCEPTANCE TEST — 14 CRITERIA

The build is complete when all 14 pass. Do not mark complete until every one passes.

1. Click Force Sync. Within 30 seconds, Slate Board shows real player names with real lines from PrizePicks.
2. At least 2 props show market data — DK and FD lines both present, True Edge % calculated and displayed.
3. At least 5 NBA props show OUR PROJ value with a confidence badge (high / medium / low). Confidence badge is colored (green/amber/red).
4. An MLB prop shows a pitcher name in its projection notes field.
5. An outdoor MLB game shows a weather badge with temperature and wind. A domed stadium (TOR, TBR, MIA, ARI, TEX, HOU, MIL, SEA) shows no weather badge.
6. Run Optimizer, Goblin Hunter strategy, 4-pick Power. All 4 picks returned are Goblin OVER. EV is shown for each combination.
7. Load optimizer result into Entry Builder. Probability chain displays (e.g., 65% x 65% x 65% x 65% = 17.9%). Kelly stake is shown.
8. Add two NBA players from the same team. A correlation warning appears. The warning names the team and explains the shared possession problem.
9. Open AI Analyst. Switch to Contradiction Check mode. Click Run. Claude returns structured findings referencing real player names from today's live slate — not generic advice.
10. Mark a pick as HIT with a closing line of 27.5. CLV record appears in the CLV Tracker page. The CLV value is calculated correctly (closing line minus locked line, direction-adjusted).
11. Verify probability calibration updated after Fix 10:
Call `GET /api/debug/calibration?sport=NBA&statType=Points`
Response shows `sampleSize` incremented, `hitRate` updated, source labeled.
This endpoint is the app-verifiable proof — not a DB check.
12. Set a \$50 daily loss limit in Settings. Submit a \$50 entry and mark it as a loss. Submit another entry immediately after. The loss limit banner appears. The second entry submission is blocked or requires override confirmation.

13. User Guide page loads, is searchable, and shows a table of contents. On first load, it sets the `hasSeenGuide` `localStorage` flag.
 14. Streak Tracker page loads. If no results have been logged yet, it shows an explicit empty state: "No streaks yet – streaks build as you log pick results."
-

WHAT SUCCESS LOOKS LIKE

After 6 months of consistent use:

- CLV trending positive on the 90-day chart
- Probability calibration showing "Your Data" labels instead of "Default Estimate" on most props
- Weekly learning report identifying which sports and stat types the user evaluates best
- Behavioral log showing reduced tilt patterns over time
- Manual override hit rate approaching or exceeding optimizer hit rate

The build delivers the infrastructure. The user builds the edge by running it consistently and trusting the process over short-term variance.